TER Project: SHACL Shapes Generator and Validator

Done by: Salma Alkhalily

Supervised by: Dr. Fatiha Sais

Abstract

This project uses a tool to generate keys for RDF data files and uses these keys to create SHACL shapes for validation purposes. The generated SHACL shapes are validated against the original data file and other data files that share the same keys.

Two scenarios are done in this project. In the first scenario, SHACL shapes are directly generated from the keys and validated against the RDF data file. In the second scenario, there is a mapping file that contains equivalent keys corresponding to the generated ones. SHACL shapes are generated for these equivalent keys and validated against the data file. This approach enables validation across multiple data files with the same key structures but different key names.

The workflows were implemented using Jena, known for its robust semantic data processing capabilities, and a user-friendly interface developed with JavaFX and CSS a little bit. This report provides a detailed explanation of each scenario, including their methodologies, supported by diagrams and illustrations.

Introduction

RDF (Resource Description Framework) datasets grow in complexity, so maintaining their integrity becomes challenging, particularly when integrating data from diverse sources. SHACL (Shapes Constraint Language) is a powerful tool for validating RDF data against defined schemas, ensuring conformance to required structures.

This project focuses on using the generated keys -by a tool- derived from a data file to generate SHACL Shapes and validate them against the data files, and for this there is two scenarios are enhanced in this project:

1. **Without equivalent SHACL shapes generation validation:** using the original names for the generated keys.
2. **With equivalent SHACL shapes generation validation:** using a mapping file that contains the pre-defined equivalent names for the generated keys.

The workflows are implemented using Jena, a robust semantic web framework, and JavaFX with CSS for developing a user-friendly interface.

This report details the methodologies and the results for the two scenarios, supported by diagrams and images from the implementation.

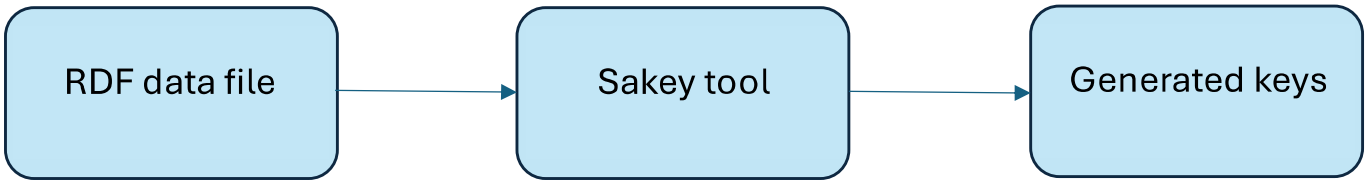
Approach and Implementation

There are two implemented scenarios:

1. **Without equivalent SHACL shapes generation validation.**
2. **With equivalent SHACL shapes generation validation:** this scenario is important when we have two data files that have the same keys but with different names.

The implementation is divided into three parts, the first one is common between the two scenarios and the other ones are different for each scenario, everything will be described in detail in this section.

❖ **Part1: Generate Keys for RDF File:** This part is shared between the two scenarios, here is the process:



➢ **<u>The input:</u>** is an RDF data file that must be in a N-Triples format (.nt), for example:

```
<http://www.okkam.org/oaie/restaurant1-Restaurant56> <http://www.okkam.org/ontology_restaurant1.owl#phone_number> "718/522-5200" .
Or
http://www.okkam.org/oaie/restaurant1-Restaurant56      http://www.okkam.org/ontology_restaurant1.owl#phone_number      718/522-5200
```

➢ **<u>Processing:</u>** use the sakey tool:

- SAKey: Scalable Almost Key Discovery in RDF Data File.
- **Definition:** SAKey is an approach that discovers keys in RDF data in an efficient way. To prune the search space, SAKey exploits characteristics of the data that are dynamically detected during the process. Furthermore, this approach can discover keys in datasets where erroneous data or duplicates exist (i.e., almost keys). The approach has been evaluated on different synthetic and real datasets. The results

show both the relevance of almost keys and the efficiency of discovering them.

- SAKey takes as input two parameters:
  a) the file where you want the key discovery to take place.
  b) the number of exceptions n: the n will lead to the discovery of n-non keys and then the extraction of (n-1)-almost keys.
- This command is to run the sakey tool:

"java -jar "jar_fullpath"/SAKey.jar "file_fullpath" n"

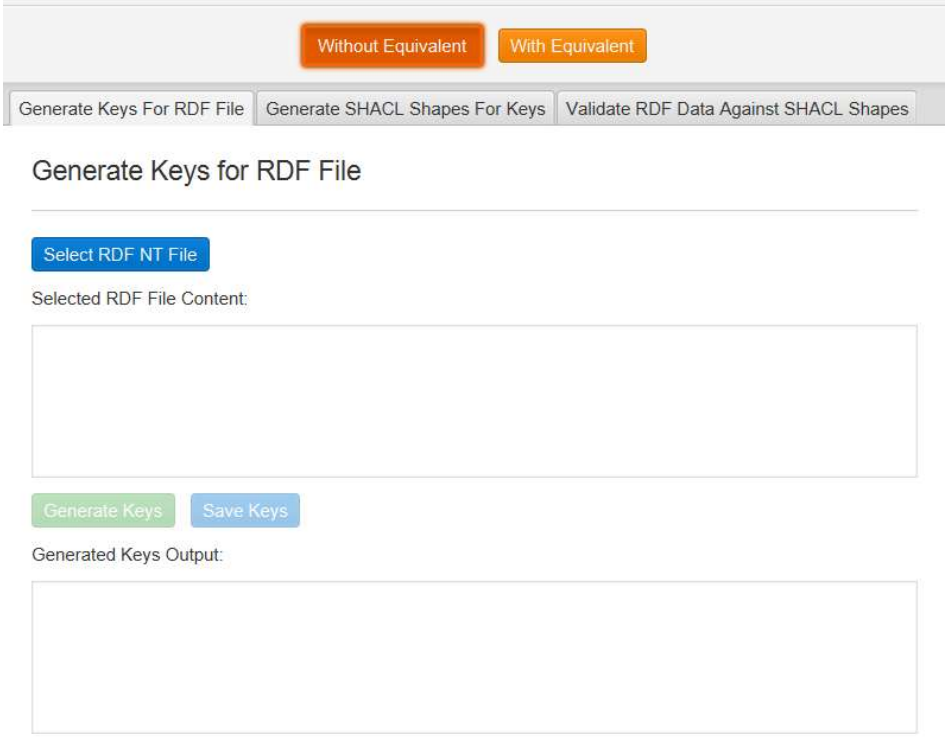
For example, in the following command:

java -jar /Users/salma/SAKey.jar /Users/salma/datasets/DB_Lake.nt 1

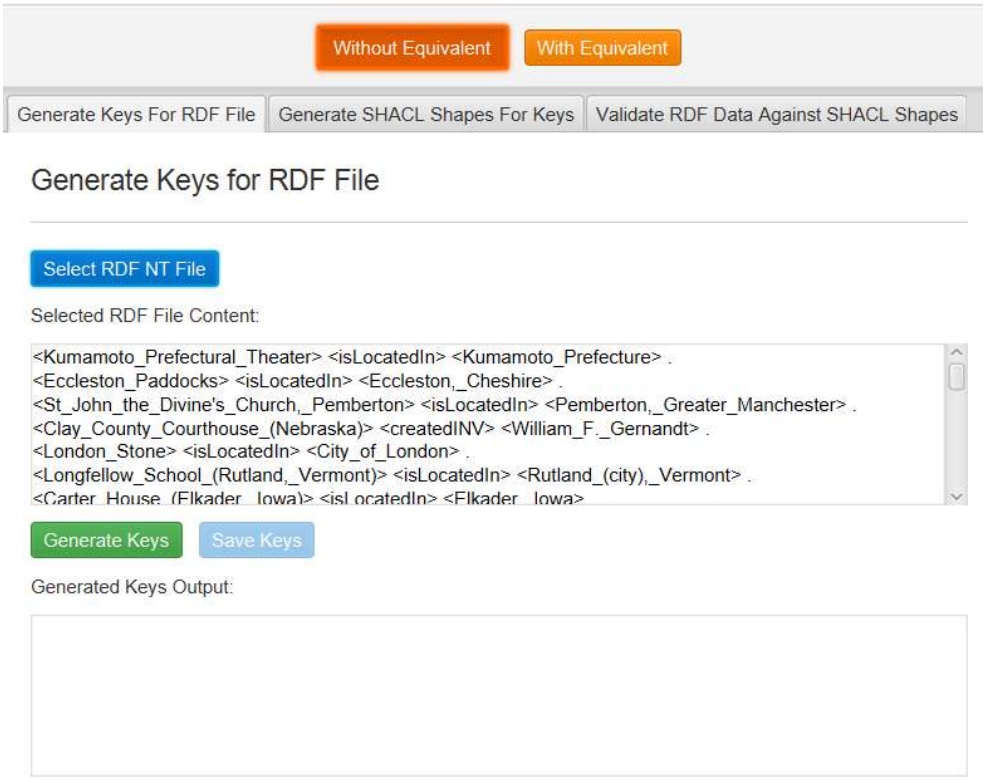
In this example, SAKey will discover first 1-non keys and then derive 0-almost keys for the file /Users/salma/datasets/DB_Lake.nt.

➢ **The output:** is the generated keys by the sakey tool (n-non keys & n-1-almost keys).

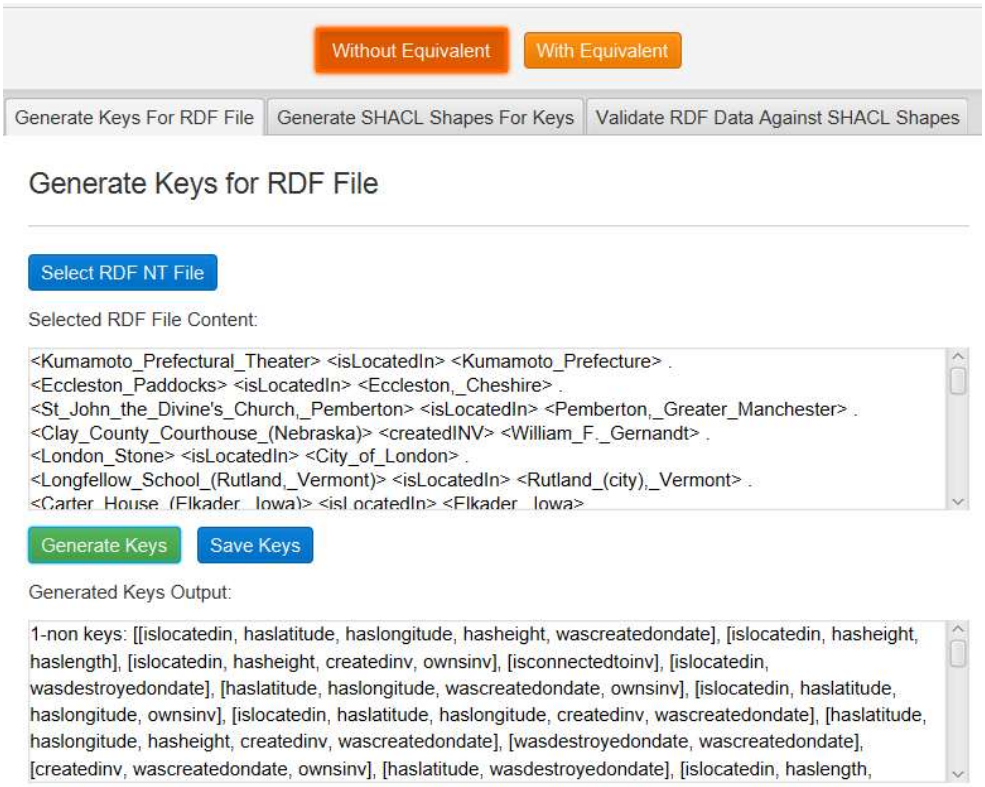
✓ **Here is a full example about this part:**



Initial interface for Generate Keys for RDF File



After selecting an RDF NT Data File and show its content



After selecting Generate Keys (run sakey tool)

```
1-non keys: [[islocatedin, haslatitude, haslongitude, hasheight, wascreatedondate], [islocatedin, hasheight, haslength], [islocatedin,
hasheight, createdinv, ownsinv], [isconnectedtoinv], [islocatedin, wasdestroyedondate], [haslatitude, haslongitude, wascreatedondate,
ownsinv], [islocatedin, haslatitude, haslongitude, ownsinv], [islocatedin, haslatitude, haslongitude, createdinv, wascreatedondate],
[haslatitude, haslongitude, hasheight, createdinv, wascreatedondate], [wasdestroyedondate, wascreatedondate], [createdinv,
wascreatedondate, ownsinv], [haslatitude, wasdestroyedondate], [islocatedin, haslength, createdinv], [islocatedin, haslatitude,
haslength], [haslatitude, haslongitude, hasheight, createdinv, ownsinv]]

0-almost keys:[[haslatitude, hasheight, haslength], [islocatedininv], [wasdestroyedondate, haslongitude], [hasheight, haslength,
createdinv], [haslongitude, createdinv, wascreatedondate, ownsinv], [wasdestroyedondate, haslength], [isconnectedto], [haslongitude,
haslength], [hasduration], [hasheight, wascreatedondate, ownsinv], [haslatitude, createdinv, wascreatedondate, ownsinv], [haslength,
wascreatedondate], [islocatedin, haslongitude, hasheight, ownsinv], [haslatitude, isconnectedtoinv], [islocatedin, haslatitude,
createdinv, ownsinv], [wasdestroyedondate, hasheight], [createdinv, isconnectedtoinv], [haspages], [haslength, isconnectedtoinv],
[wasdestroyedondate, isconnectedtoinv], [wasdestroyedondate, ownsinv], [haslength, ownsinv], [islocatedin, haslatitude, hasheight,
createdinv], [islocatedin, wasdestroyedondate, wascreatedondate], [haslatitude, haslength, createdinv], [islocatedin,
isconnectedtoinv], [islocatedin, hasheight, createdinv, wascreatedondate], [islocatedin, wascreatedondate, ownsinv], [haslongitude,
isconnectedtoinv], [participatedininv], [islocatedin, haslongitude, createdinv, ownsinv], [isconnectedtoinv, wascreatedondate],
[haslatitude, wasdestroyedondate, wascreatedondate], [islocatedin, haslatitude, hasheight, ownsinv], [hasheight, isconnectedtoinv],
[islocatedin, haslongitude, hasheight, createdinv], [wasdestroyedondate, createdinv], [happenedininv], [islocatedin, haslatitude,
wasdestroyedondate], [isconnectedtoinv, ownsinv]]
```

After saving the generated keys in txt file

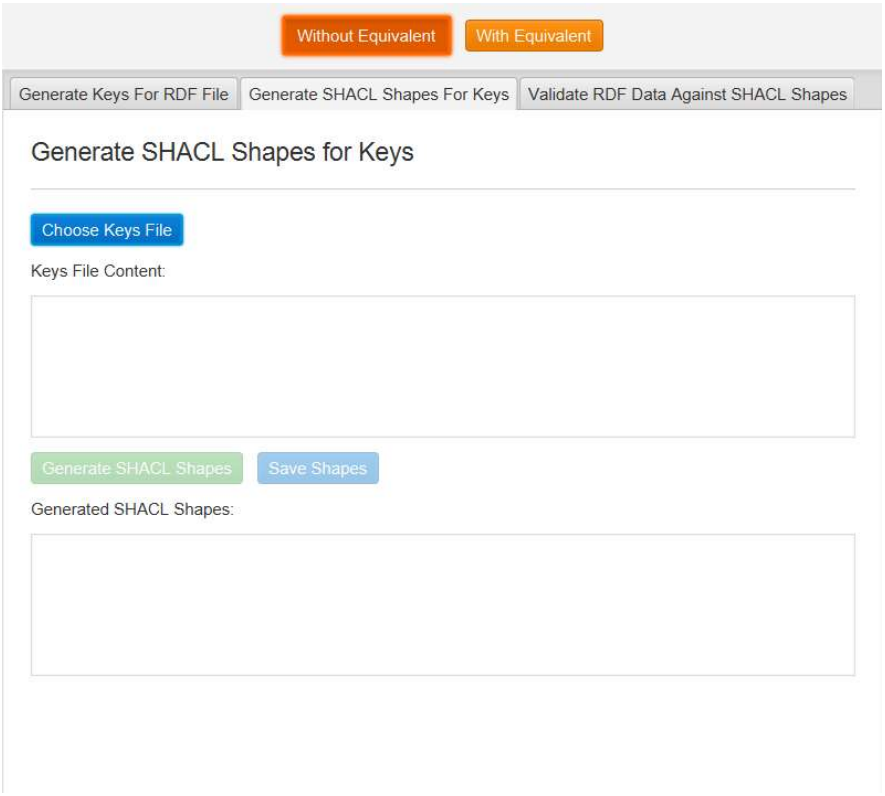
- **Part 2: Generate SHACL Shapes for Keys:** The generated shapes are used for validating RDF data and ensuring unique constraints through SPARQL constraints and rules.

- **<u>Generating SHACL Shapes:</u>** for each almost key define shape, for each shape define:
  - **Property constraints:** it must be a string; it can have a maximum of one value per subject and the name of the property being validated.
  - **Duplication Constraint:** this constraint is to ensure that each node has a different value for each almost key (for each almost key, no duplicated values between two nodes). If a duplicate is detected, a violation will be triggered "Duplicate detected for properties: islocatedininv".
  - **SHACL triple rule:** if two resources share the same value for an almost key, this asserts that the first resource is same as the second one. If this happened, the nodes that share the value should be linked by owl:sameAs, otherwise a violation will be triggered "Duplicate with the same properties must be marked as owl:sameAs.".

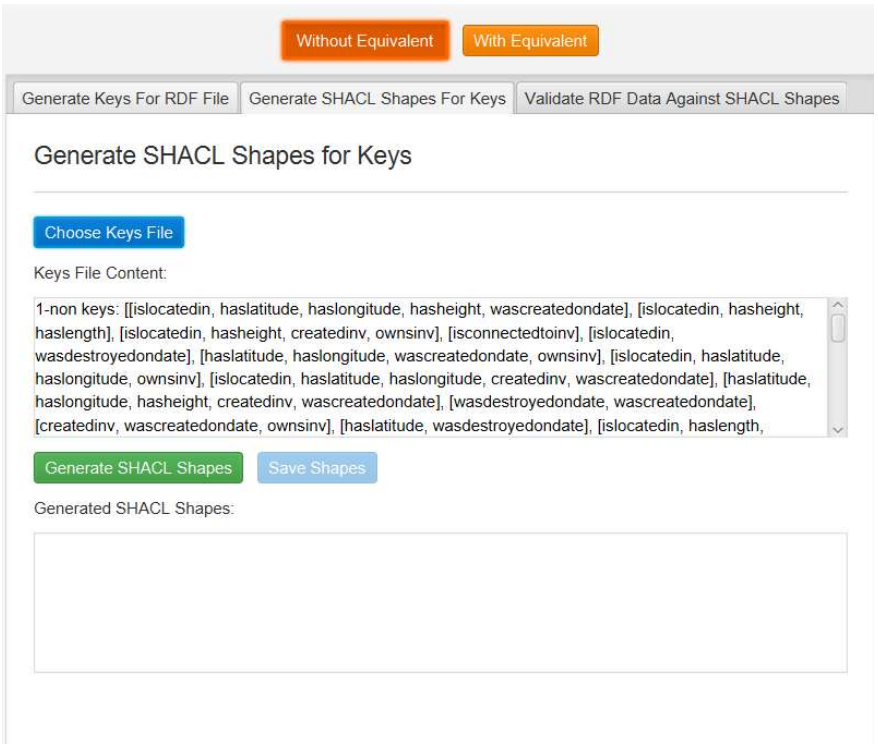
This part is different for each scenario:

- **<u>Without equivalent:</u>** get keys file as input and generate SHACL Shapes for each almost key.

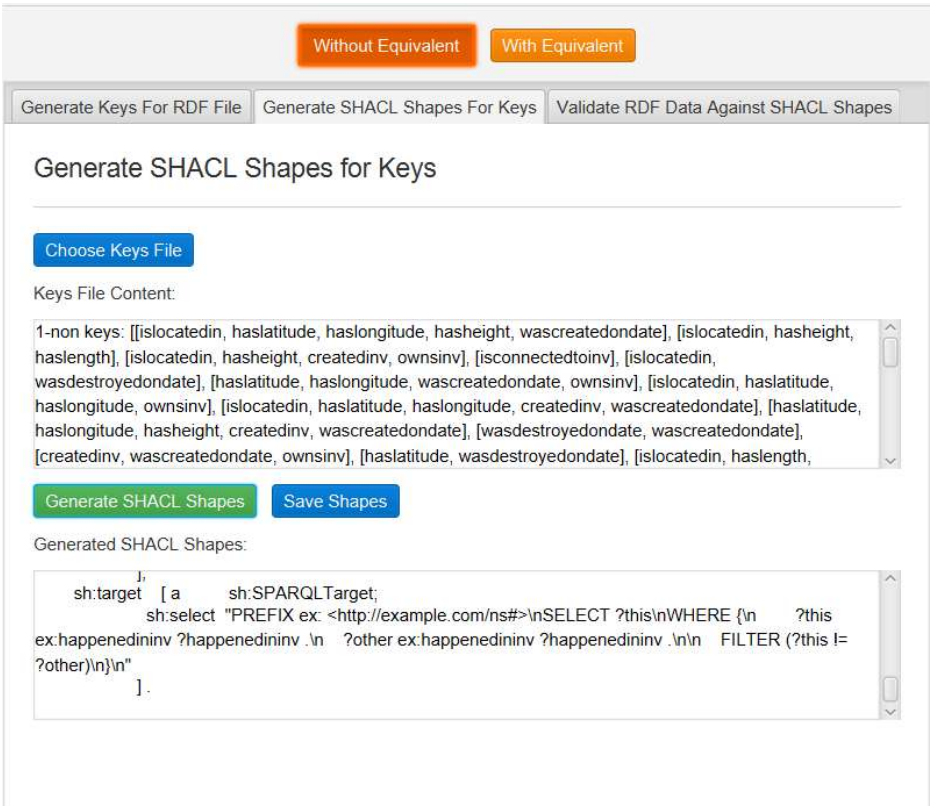
✓ **<u>Here is a full example:</u>**



First interface to generate SHACL Shapes for keys



After selecting keys file



After generating SHACL Shapes for almost keys

```
PREFIX ex:  <http://example.com/ns#>
PREFIX owl: <http://www.w3.org/2002/07/owl#>
PREFIX sh:  <http://www.w3.org/ns/shacl#>
PREFIX xsd: <http://www.w3.org/2001/XMLSchema#>

ex:Shape20  a        sh:NodeShape;
        sh:property  [ sh:datatype  xsd:string;
                       sh:maxCount  "1"^^xsd:int;
                       sh:path      ex:isconnectedtoinv
                     ];
        sh:property  [ sh:datatype  xsd:string;
                       sh:maxCount  "1"^^xsd:int;
                       sh:path      ex:wasdestroyedondate
                     ];
        sh:rule      [ a          sh:TripleRule;
                       sh:select  "PREFIX ex: <http://example.com/ns#>\nPREFIX owl: <http://www.w3.org/2002/07/owl#>\nCONSTRUCT {\n    ?this owl:sameAs ?other .\n}\nWHERE {\n        ?this ex:wasdestroyedondate ?wasdestroyedondate .\n    ?other ex:wasdestroyedondate ?wasdestroyedondate .\n    ?this ex:isconnectedtoinv ?isconnectedtoinv .\n    ?other ex:isconnectedtoinv ?isconnectedtoinv .\n\n    FILTER (?this != ?other)\n}\n"
                     ];
        sh:severity  sh:Violation;
        sh:sparql    [ a           sh:SPARQLConstraint;
                       sh:message  "Duplicate with the same properties must be marked as owl:sameAs.";
                       sh:select   "PREFIX ex: <http://example.com/ns#>\nPREFIX owl: <http://www.w3.org/2002/07/owl#>\nSELECT ?this ?other\nWHERE {\n        ?this ex:wasdestroyedondate ?wasdestroyedondate .\n    ?other ex:wasdestroyedondate ?wasdestroyedondate .\n    ?this ex:isconnectedtoinv ?isconnectedtoinv .\n    ?other ex:isconnectedtoinv ?isconnectedtoinv .\n\n    FILTER (?this != ?other)\n    FILTER NOT EXISTS { ?this owl:sameAs ?other }\n}\n"
                     ];
        sh:sparql    [ a           sh:SPARQLConstraint;
                       sh:message  "Duplicate detected for properties: wasdestroyedondate, isconnectedtoinv";
```

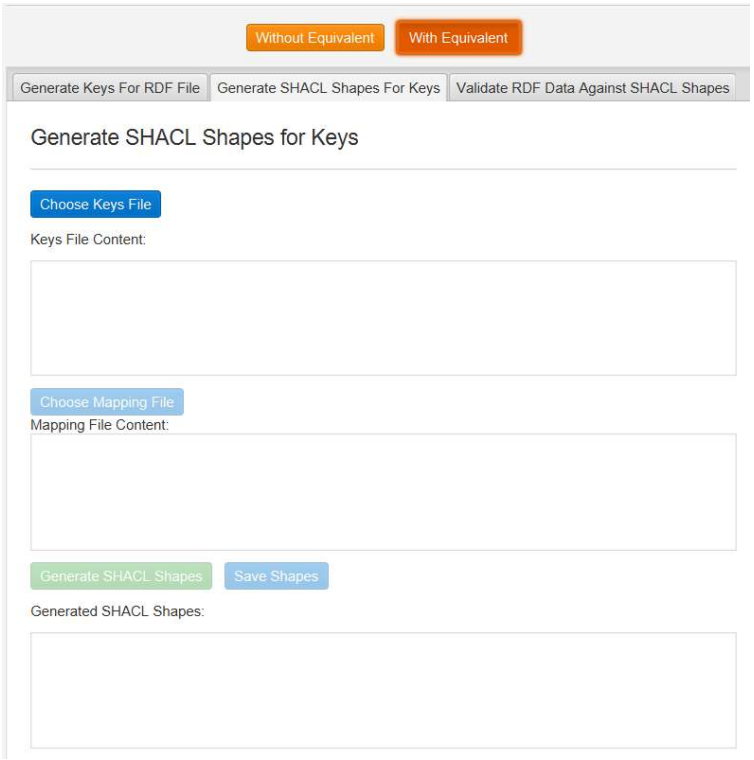
Generated SHACL Shapes file content

```
ex:Shape2  a         sh:NodeShape;
        sh:property  [ sh:datatype  xsd:string;
                       sh:maxCount  "1"^^xsd:int;
                       sh:path      ex:islocatedininv
                     ];
        sh:rule      [ a          sh:TripleRule;
                       sh:select  "PREFIX ex: <http://example.com/ns#>\nPREFIX owl: <http://www.w3.org/2002/07/owl#>\nCONSTRUCT
{\n    ?this owl:sameAs ?other .\n}\nWHERE {\n        ?this ex:islocatedininv ?islocatedininv .\n    ?other ex:islocatedininv ?
islocatedininv .\n\n    FILTER (?this != ?other)\n}\n"
                     ];
        sh:severity  sh:Violation;
        sh:sparql    [ a           sh:SPARQLConstraint;
                       sh:message  "Duplicate with the same properties must be marked as owl:sameAs.";
                       sh:select   "PREFIX ex: <http://example.com/ns#>\nPREFIX owl: <http://www.w3.org/2002/07/owl#>\nSELECT ?this ?
other\nWHERE {\n        ?this ex:islocatedininv ?islocatedininv .\n    ?other ex:islocatedininv ?islocatedininv .\n\n    FILTER (?
this != ?other)\n    FILTER NOT EXISTS { ?this owl:sameAs ?other }\n}\n"
                     ];
        sh:sparql    [ a           sh:SPARQLConstraint;
                       sh:message  "Duplicate detected for properties: islocatedininv";
                       sh:select   "PREFIX ex: <http://example.com/ns#>\nSELECT ?this\nWHERE {\n        ?this ex:islocatedininv ?
islocatedininv .\n    ?other ex:islocatedininv ?islocatedininv .\n\n    FILTER (?this != ?other)\n}\n"
                     ];
        sh:target    [ a          sh:SPARQLTarget;
                       sh:select  "PREFIX ex: <http://example.com/ns#>\nSELECT ?this\nWHERE {\n        ?this ex:islocatedininv ?
islocatedininv .\n    ?other ex:islocatedininv ?islocatedininv .\n\n    FILTER (?this != ?other)\n}\n"
                     ] .
```

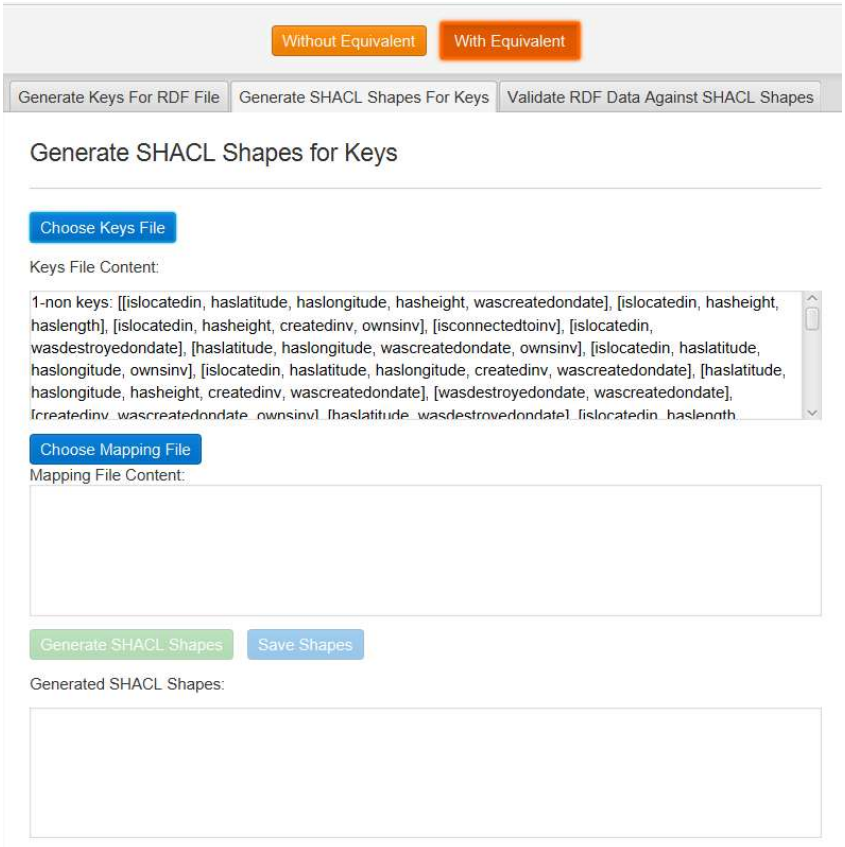
SHACL Shape for [islocatedininv] almost key

- **With equivalent:** get keys file and mapping file as input and generate SHACL Shapes for each almost key but with the equivalent names for keys in the mapping file.

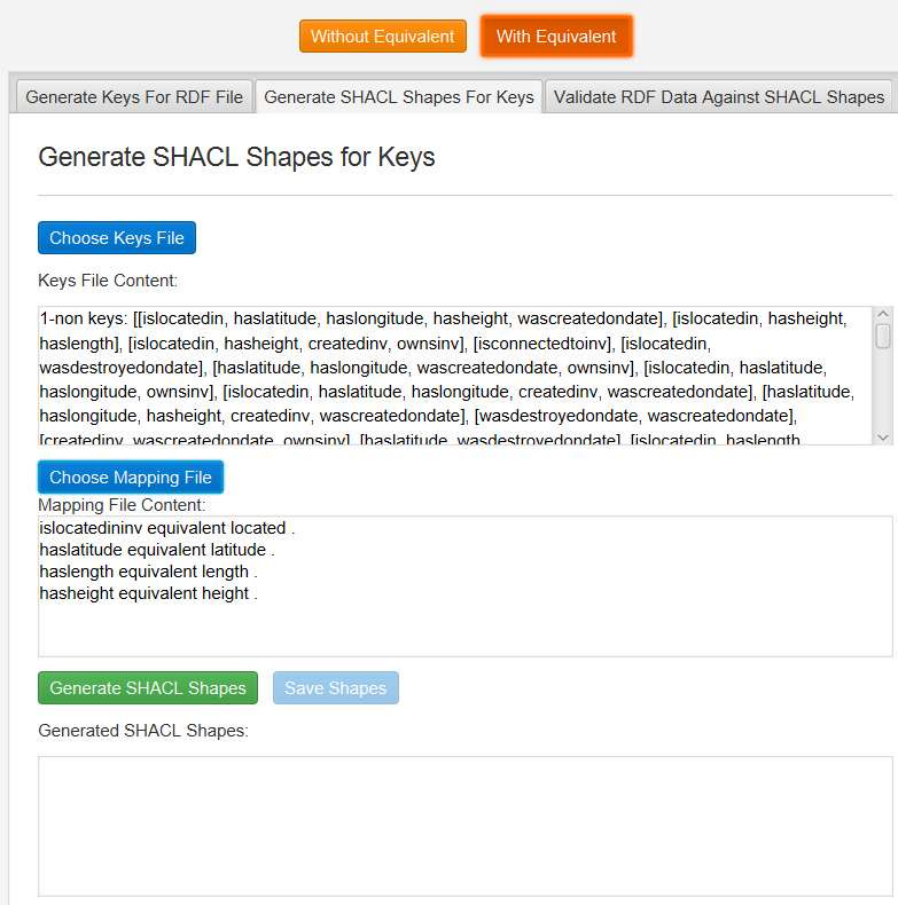
✓ **Here is a full example about this part:**



First interface to generate SHACL Shapes for keys using equivalent names



After selecting keys file

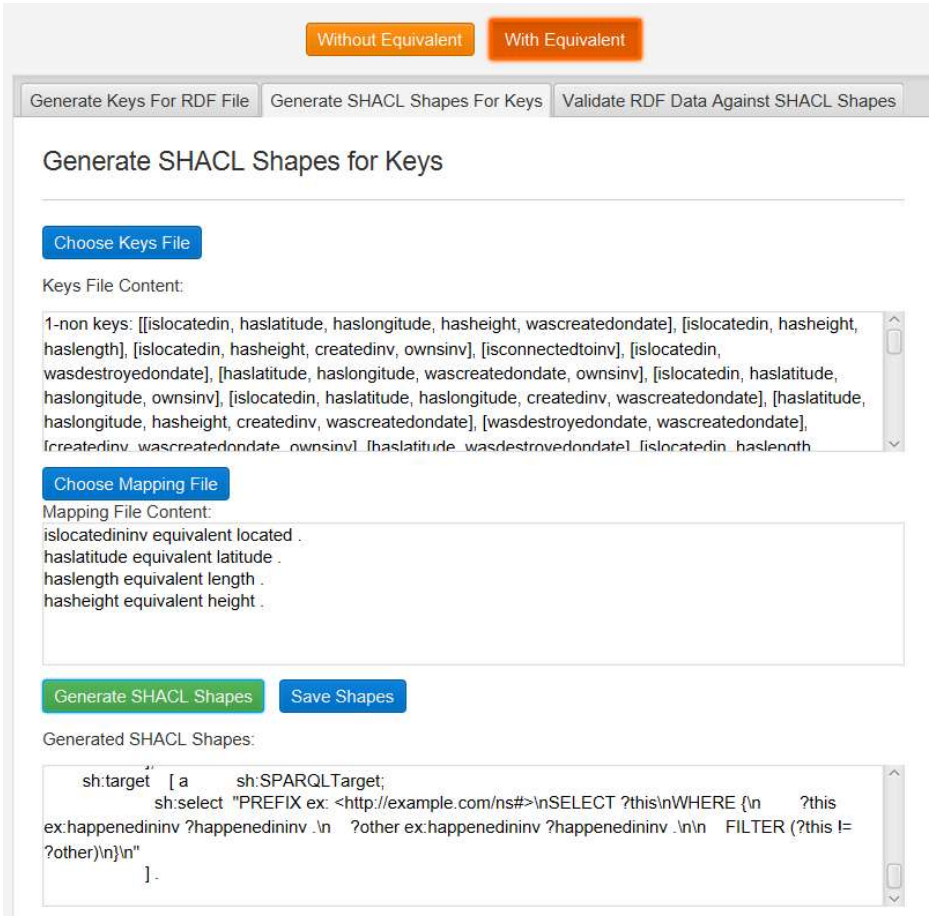


After selecting mapping file

```
islocatedininv equivalent located .
haslatitude equivalent latitude .
haslength equivalent length .
hasheight equivalent height .
```

Mapping file content example

Note: here just for testing I selected the first two almost keys and write the equivalent names for them



After generating SHACL Shapes for almost keys

```
ex:Shape2  a          sh:NodeShape;
        sh:property   [ sh:datatype  xsd:string;
                        sh:maxCount  "1"^^xsd:int;
                        sh:path      ex:located
                      ];
        sh:rule       [ a           sh:TripleRule;
                        sh:select   "PREFIX ex: <http://example.com/ns#>\nPREFIX owl: <http://www.w3.org/2002/07/owl#>\nCONSTRUCT
{\n    ?this owl:sameAs ?other .\n}\nWHERE {\n        ?this ex:located ?located .\n    ?other ex:located ?located .\n\n    FILTER (?
this != ?other)\n}\n"
                      ];
        sh:severity   sh:Violation;
        sh:sparql     [ a           sh:SPARQLConstraint;
                        sh:message  "Duplicate with the same properties must be marked as owl:sameAs.";
                        sh:select   "PREFIX ex: <http://example.com/ns#>\nPREFIX owl: <http://www.w3.org/2002/07/owl#>\nSELECT ?this ?
other\nWHERE {\n        ?this ex:located ?located .\n    ?other ex:located ?located .\n\n    FILTER (?this != ?other)\n    FILTER NOT
EXISTS { ?this owl:sameAs ?other }\n}\n"
                      ];
        sh:sparql     [ a           sh:SPARQLConstraint;
                        sh:message  "Duplicate detected for properties: located";
                        sh:select   "PREFIX ex: <http://example.com/ns#>\nSELECT ?this\nWHERE {\n        ?this ex:located ?located .
\n    ?other ex:located ?located .\n\n    FILTER (?this != ?other)\n}\n"
                      ];
        sh:target     [ a           sh:SPARQLTarget;
                        sh:select   "PREFIX ex: <http://example.com/ns#>\nSELECT ?this\nWHERE {\n        ?this ex:located ?located .
\n    ?other ex:located ?located .\n\n    FILTER (?this != ?other)\n}\n"
                      ] .
```

Gener SHACL Shape for [islocatedininv] almost key with its equivalent name" located"

❖ **Part 3: Validate RDF data against SHACL Shapes:** generate validation report including the number of violations for each node and the total number of violations.

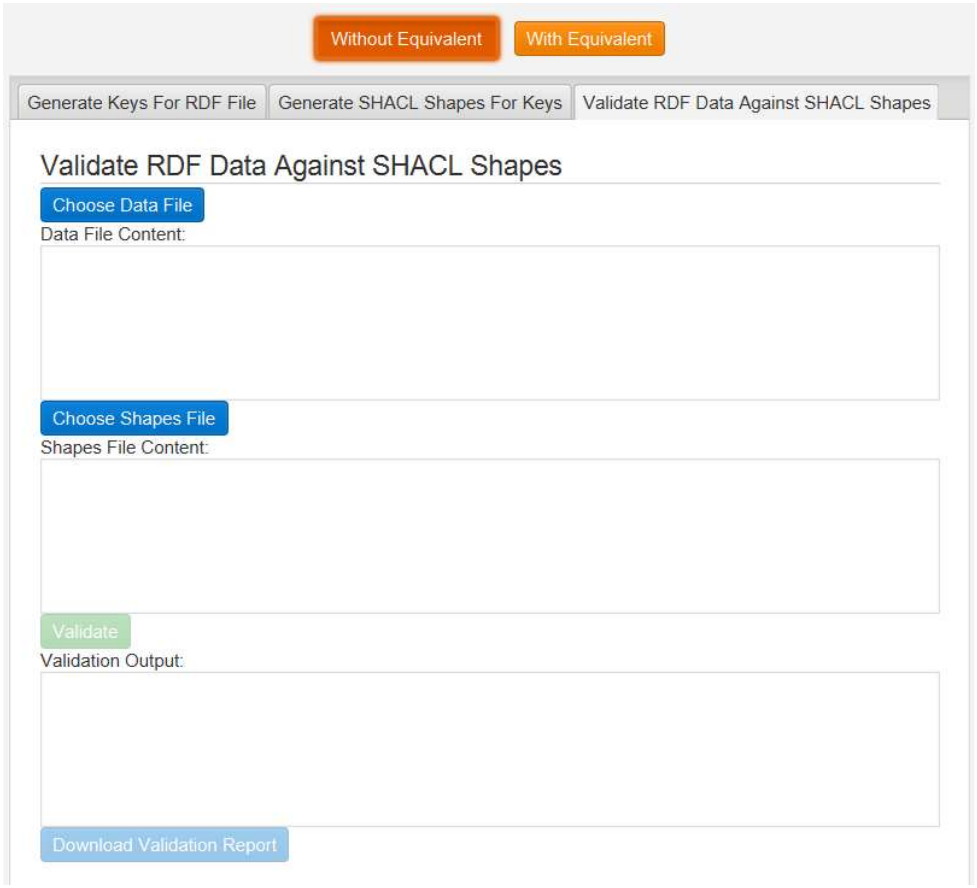
➢ **<u>Validate RDF data against SHACL Shapes:</u>**

- **Process the data file:** like remove empty lines, splits each RDF triple (subject | predict| object), Transforms URIs and object literals, do replacements for some symbols, and all of them are done in a spark session.
- **Load SHACL shapes graph.**
- **Load RDF graph.**
- **Parse SHACL shapes constraints from shapes graph.**
- **Perform SHACL validation.**
- **Count total violations and the number of violations for each node.**

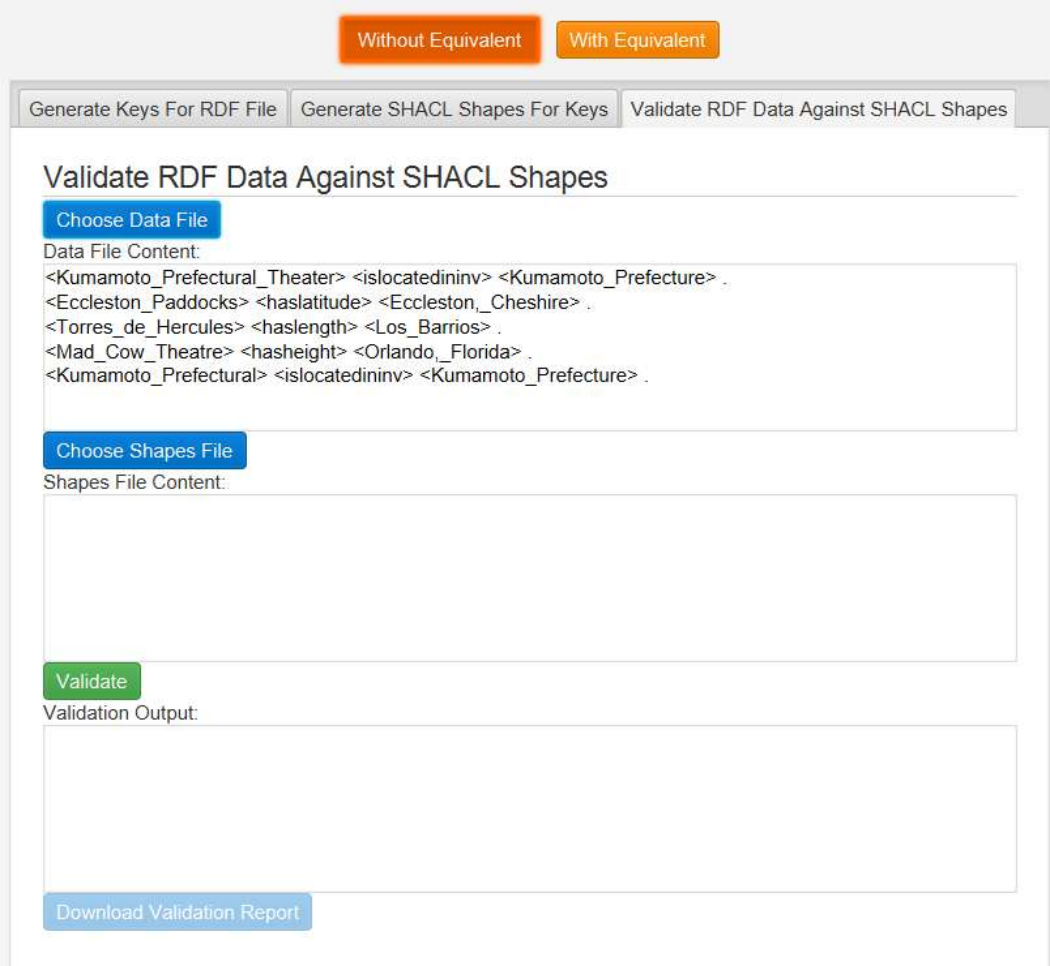
This part is different for each scenario:

➢ **<u>Without equivalent:</u>** get data file and SHACL shapes file as input, validate RDF data against SHACL Shapes and generate the validation report.

✓ **<u>Here is a full example:</u>**



First interface for validate RDF data against SHACL Shapes without equivalent

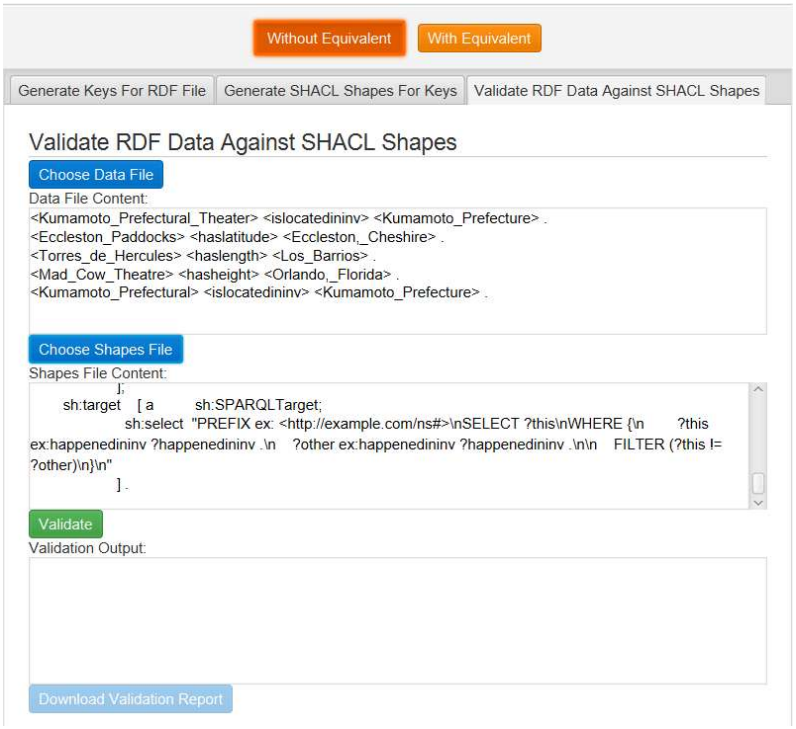


After selecting test data file

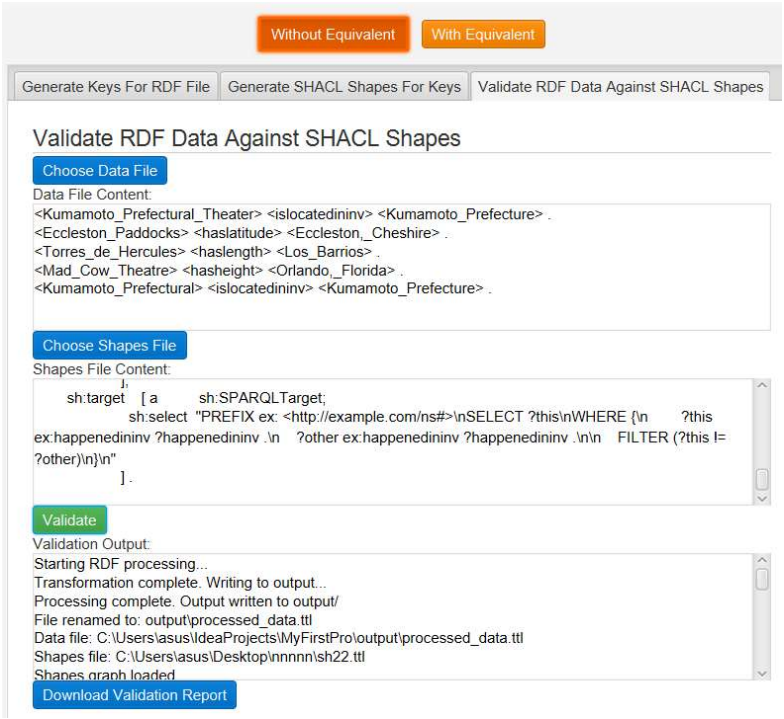
```
<Kumamoto_Prefectural_Theater> <islocatedininv> <Kumamoto_Prefecture> .
<Eccleston_Paddocks> <haslatitude> <Eccleston,_Cheshire> .
<Torres_de_Hercules> <haslength> <Los_Barrios> .
<Mad_Cow_Theatre> <hasheight> <Orlando,_Florida> .
<Kumamoto_Prefectural> <islocatedininv> <Kumamoto_Prefecture> .
```

Test data file content

Note: here I created a small data file just for testing



After selecting shapes file



After doing validation

Starting RDF processing...

Transformation complete. Writing to output...

Processing complete. Output written to output/

File renamed to: output\processed_data.ttl

Data file: C:\Users\asus\IdeaProjects\MyFirstPro\output\processed_data.ttl

Shapes file: C:\Users\asus\Desktop\nnnnn\sh22.ttl

Shapes graph loaded.

Validation report generated.

Report: [Node=<http://example.com/ns#Kumamoto_Prefectural_Theater>

Duplicate detected for properties: islocatedininv,
Node=<http://example.com/ns#Kumamoto_Prefectural_Theater>

Duplicate with the same properties must be marked as owl:sameAs.,
Node=<http://example.com/ns#Kumamoto_Prefectural>

Duplicate detected for properties: islocatedininv,
Node=<http://example.com/ns#Kumamoto_Prefectural>

Duplicate with the same properties must be marked as owl:sameAs.]

RDF Validation Model:

_:Bcadc6b06X2D91dbX2D408fX2D9742X2Dbdaf77b6189b
<http://www.w3.org/ns/shacl#conforms> "false"^^<http://www.w3.org/2001/XMLSchema#boolean>
.

_:Bcadc6b06X2D91dbX2D408fX2D9742X2Dbdaf77b6189b <http://www.w3.org/ns/shacl#result>
_:B05551600X2D32acX2D4dfdX2D98f2X2De5e24c67174a .

_:Bcadc6b06X2D91dbX2D408fX2D9742X2Dbdaf77b6189b <http://www.w3.org/ns/shacl#result>
_:B85760273X2D09a7X2D4933X2D8197X2Dac73e511991a .

_:Bcadc6b06X2D91dbX2D408fX2D9742X2Dbdaf77b6189b <http://www.w3.org/ns/shacl#result>
_:B417ab7a9X2D3cb6X2D4566X2D9ce8X2D01adb904c9d3 .

_:Bcadc6b06X2D91dbX2D408fX2D9742X2Dbdaf77b6189b <http://www.w3.org/ns/shacl#result>
_:Bfe03487fX2DeeacX2D40e3X2D9119X2Dc5d44d9c4dc5 .

_:Bcadc6b06X2D91dbX2D408fX2D9742X2Dbdaf77b6189b <http://www.w3.org/1999/02/22-rdf-
syntax-ns#type> <http://www.w3.org/ns/shacl#ValidationReport> .

_:B417ab7a9X2D3cb6X2D4566X2D9ce8X2D01adb904c9d3 <http://www.w3.org/ns/shacl#value>
<http://example.com/ns#Kumamoto_Prefectural_Theater> .

_:B417ab7a9X2D3cb6X2D4566X2D9ce8X2D01adb904c9d3

Validation report first page

<http://www.w3.org/ns/shacl#sourceShape> <http://example.com/ns#Shape2> .

_:B417ab7a9X2D3cb6X2D4566X2D9ce8X2D01adb904c9d3
<http://www.w3.org/ns/shacl#resultMessage> "Duplicate with the same properties must be marked as owl:sameAs." .

_:B417ab7a9X2D3cb6X2D4566X2D9ce8X2D01adb904c9d3
<http://www.w3.org/ns/shacl#sourceConstraintComponent>
<http://www.w3.org/ns/shacl#SPARQLConstraintComponent> .

_:B417ab7a9X2D3cb6X2D4566X2D9ce8X2D01adb904c9d3
<http://www.w3.org/ns/shacl#resultSeverity> <http://www.w3.org/ns/shacl#Violation> .

_:B417ab7a9X2D3cb6X2D4566X2D9ce8X2D01adb904c9d3
<http://www.w3.org/ns/shacl#focusNode>
<http://example.com/ns#Kumamoto_Prefectural_Theater> .

_:B417ab7a9X2D3cb6X2D4566X2D9ce8X2D01adb904c9d3 <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://www.w3.org/ns/shacl#ValidationResult> .

_:B05551600X2D32acX2D4dfdX2D98f2X2De5e24c67174a <http://www.w3.org/ns/shacl#value>
<http://example.com/ns#Kumamoto_Prefectural> .

_:B05551600X2D32acX2D4dfdX2D98f2X2De5e24c67174a
<http://www.w3.org/ns/shacl#sourceShape> <http://example.com/ns#Shape2> .

_:B05551600X2D32acX2D4dfdX2D98f2X2De5e24c67174a
<http://www.w3.org/ns/shacl#resultMessage> "Duplicate with the same properties must be marked as owl:sameAs." .

_:B05551600X2D32acX2D4dfdX2D98f2X2De5e24c67174a
<http://www.w3.org/ns/shacl#sourceConstraintComponent>
<http://www.w3.org/ns/shacl#SPARQLConstraintComponent> .

_:B05551600X2D32acX2D4dfdX2D98f2X2De5e24c67174a
<http://www.w3.org/ns/shacl#resultSeverity> <http://www.w3.org/ns/shacl#Violation> .

_:B05551600X2D32acX2D4dfdX2D98f2X2De5e24c67174a
<http://www.w3.org/ns/shacl#focusNode> <http://example.com/ns#Kumamoto_Prefectural> .

_:B05551600X2D32acX2D4dfdX2D98f2X2De5e24c67174a <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://www.w3.org/ns/shacl#ValidationResult> .

_:B85760273X2D09a7X2D4933X2D8197X2Dac73e511991a <http://www.w3.org/ns/shacl#value>
<http://example.com/ns#Kumamoto_Prefectural> .

_:B85760273X2D09a7X2D4933X2D8197X2Dac73e511991a
<http://www.w3.org/ns/shacl#sourceShape> <http://example.com/ns#Shape2> .

_:B85760273X2D09a7X2D4933X2D8197X2Dac73e511991a

Validation report second page

<http://www.w3.org/ns/shacl#resultMessage> "Duplicate detected for properties: islocatedininv" .

_:B85760273X2D09a7X2D4933X2D8197X2Dac73e511991a
<http://www.w3.org/ns/shacl#sourceConstraintComponent>
<http://www.w3.org/ns/shacl#SPARQLConstraintComponent> .

_:B85760273X2D09a7X2D4933X2D8197X2Dac73e511991a
<http://www.w3.org/ns/shacl#resultSeverity> <http://www.w3.org/ns/shacl#Violation> .

_:B85760273X2D09a7X2D4933X2D8197X2Dac73e511991a
<http://www.w3.org/ns/shacl#focusNode> <http://example.com/ns#Kumamoto_Prefectural> .

_:B85760273X2D09a7X2D4933X2D8197X2Dac73e511991a <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://www.w3.org/ns/shacl#ValidationResult> .

_:Bfe03487fX2DeeacX2D40e3X2D9119X2Dc5d44d9c4dc5 <http://www.w3.org/ns/shacl#value>
<http://example.com/ns#Kumamoto_Prefectural_Theater> .

_:Bfe03487fX2DeeacX2D40e3X2D9119X2Dc5d44d9c4dc5
<http://www.w3.org/ns/shacl#sourceShape> <http://example.com/ns#Shape2> .

_:Bfe03487fX2DeeacX2D40e3X2D9119X2Dc5d44d9c4dc5
<http://www.w3.org/ns/shacl#resultMessage> "Duplicate detected for properties: islocatedininv" .

_:Bfe03487fX2DeeacX2D40e3X2D9119X2Dc5d44d9c4dc5
<http://www.w3.org/ns/shacl#sourceConstraintComponent>
<http://www.w3.org/ns/shacl#SPARQLConstraintComponent> .

_:Bfe03487fX2DeeacX2D40e3X2D9119X2Dc5d44d9c4dc5
<http://www.w3.org/ns/shacl#resultSeverity> <http://www.w3.org/ns/shacl#Violation> .

_:Bfe03487fX2DeeacX2D40e3X2D9119X2Dc5d44d9c4dc5
<http://www.w3.org/ns/shacl#focusNode>
<http://example.com/ns#Kumamoto_Prefectural_Theater> .

_:Bfe03487fX2DeeacX2D40e3X2D9119X2Dc5d44d9c4dc5 <http://www.w3.org/1999/02/22-rdf-syntax-ns#type> <http://www.w3.org/ns/shacl#ValidationResult> .

Violation Counts by Individual Constraint:

Constraint: http://www.w3.org/ns/shacl#SPARQLConstraintComponent - Duplicate with the same properties must be marked as owl:sameAs.

Violations: 2

Constraint: http://www.w3.org/ns/shacl#SPARQLConstraintComponent - Duplicate detected for properties: islocatedininv

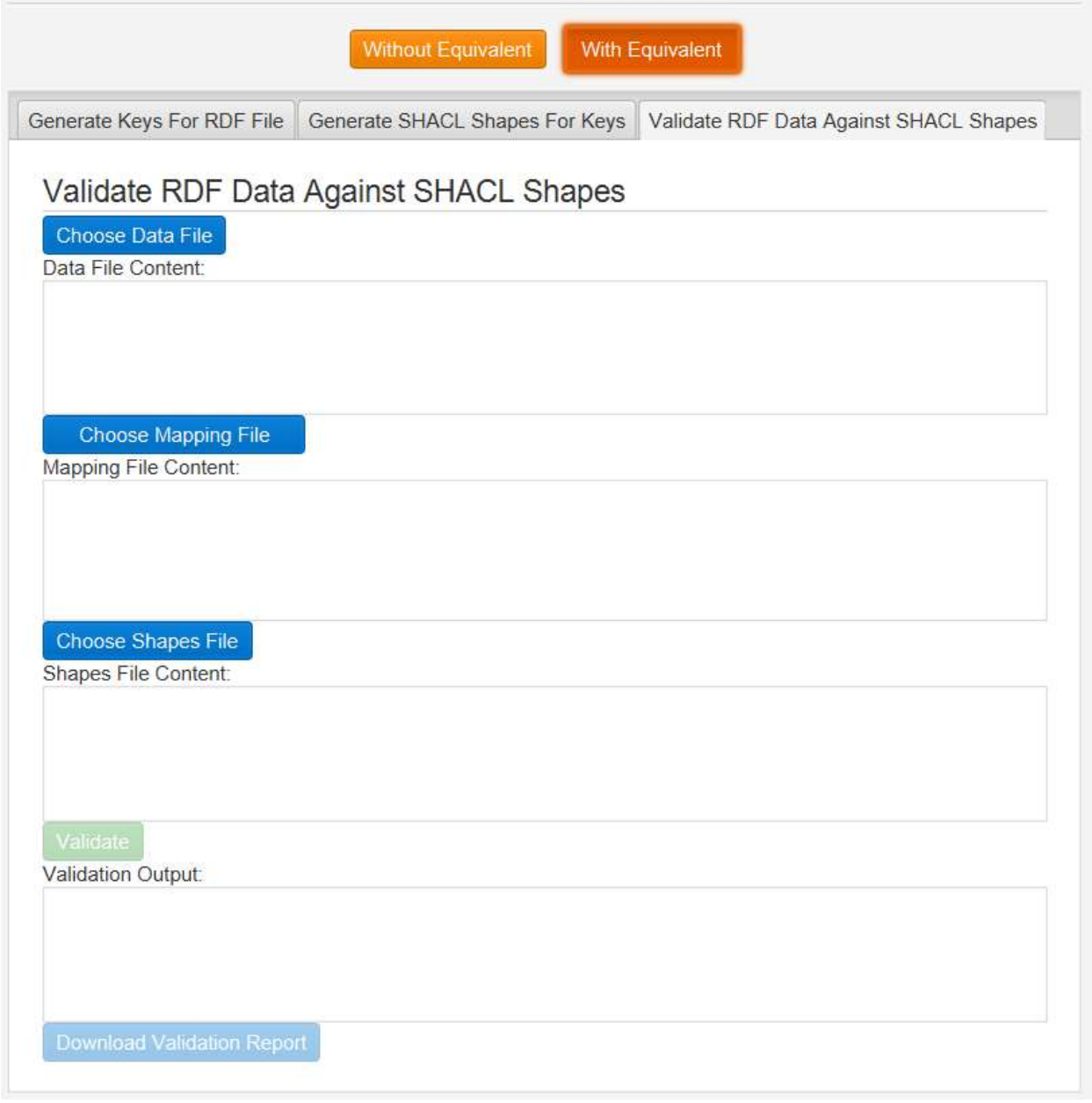
Violations: 2

Total Number of Violations: 4

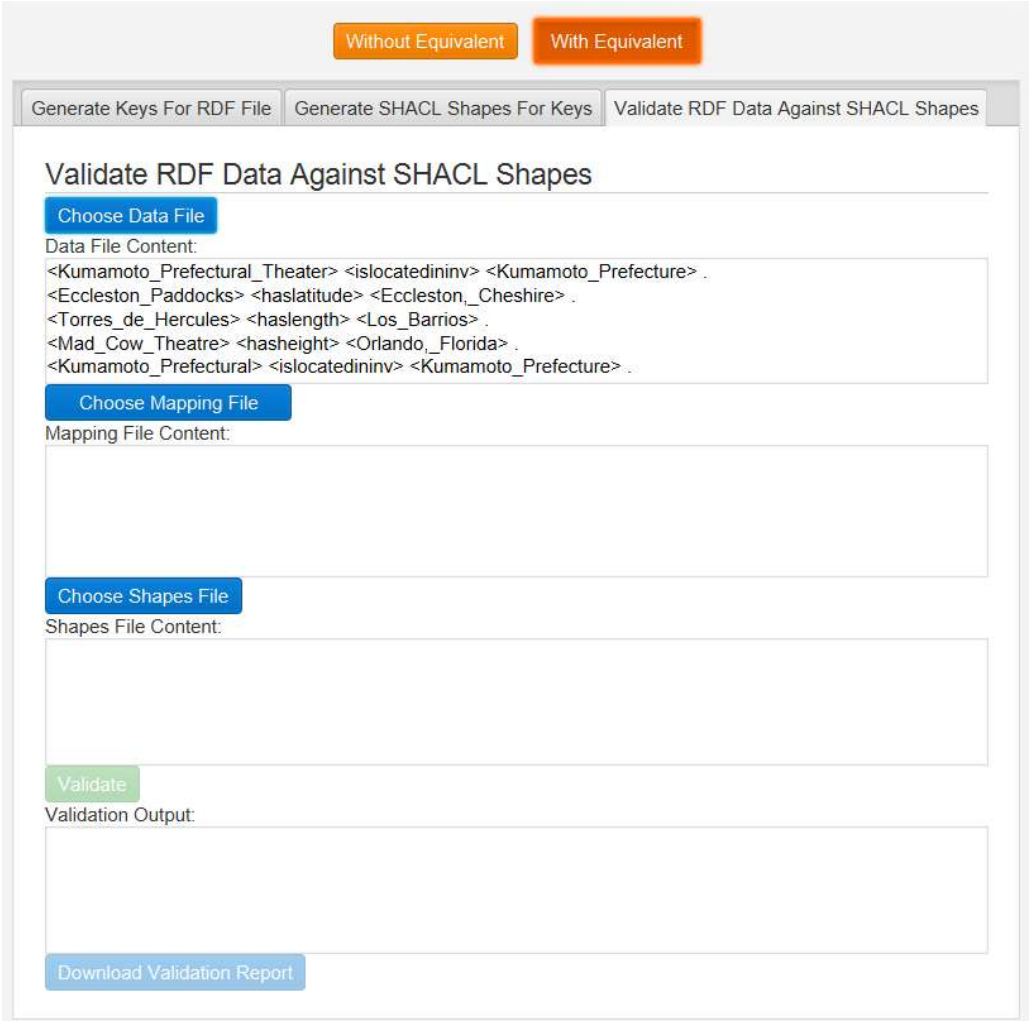
Validation report third page

In this example we have two nodes have same values for key islocatedininv, two violations for each node, one for the unique constraint and one for the sameAs rule

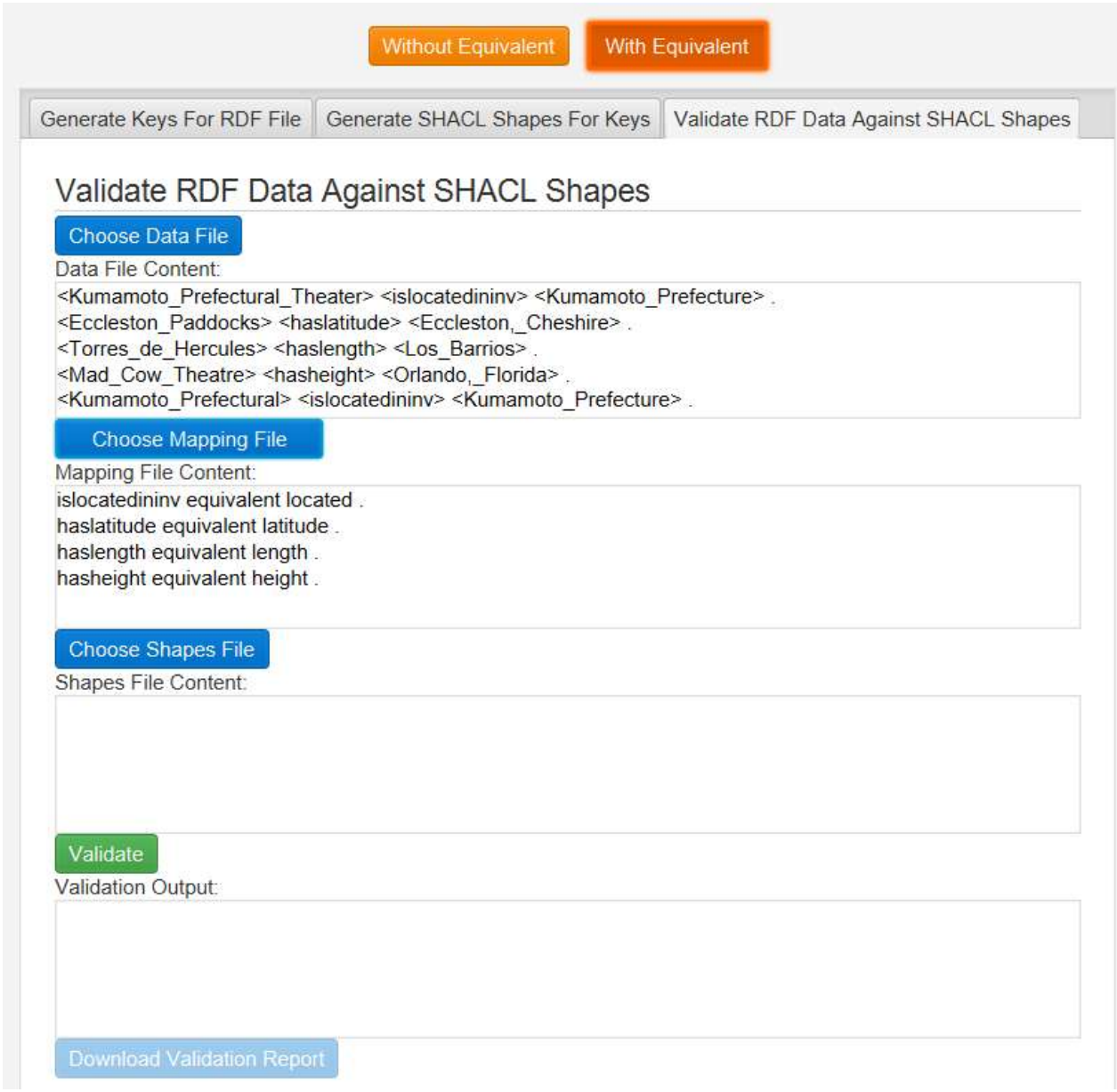
- **With equivalent:** get data file, mapping file and SHACL shapes(with equivalent names for each almost key) file as input, validate RDF data against SHACL Shapes and generate the validation report.
- ✓ **Here is a full example:**



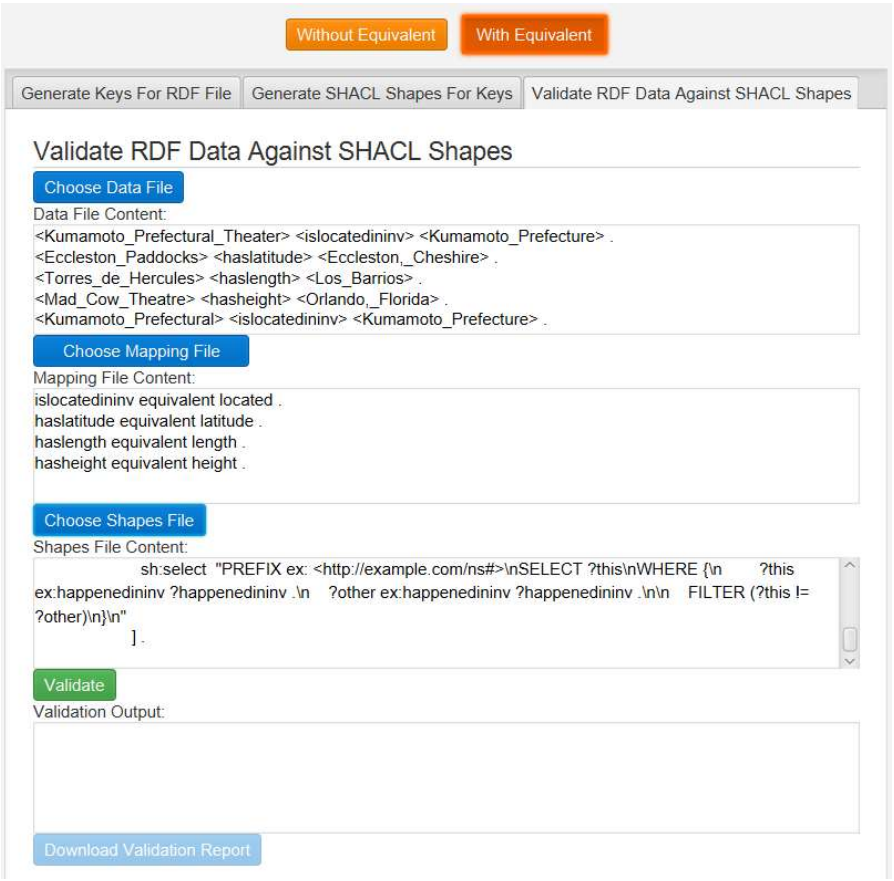
First interface for validate RDF data against SHACL Shapes with equivalent



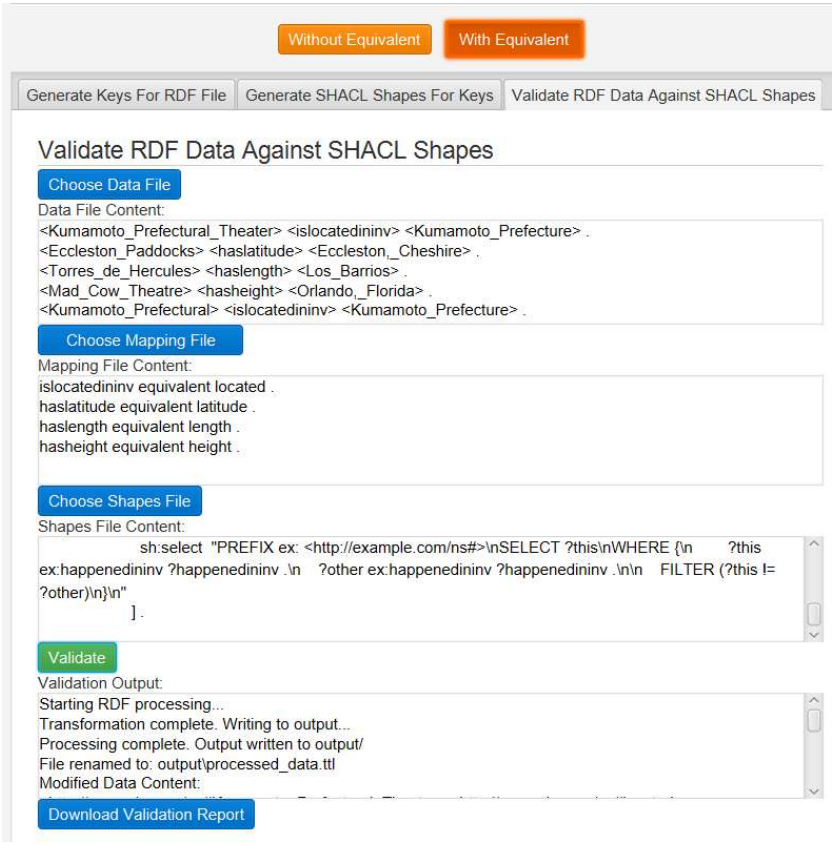
After selecting test data file



After selecting mapping file



After selecting shapes file



After doing validation

Starting RDF processing...

Transformation complete. Writing to output...

Processing complete. Output written to output/

File renamed to: output\processed_data.ttl

Modified Data Content:

<http://example.com/ns#Kumamoto_Prefectural_Theater> <http://example.com/ns#located> "Kumamoto_Prefecture " .

<http://example.com/ns#Eccleston_Paddocks> <http://example.com/ns#latitude> "Eccleston,_Cheshire " .

<http://example.com/ns#Torres_de_Hercules> <http://example.com/ns#length> "Los_Barrios " .

<http://example.com/ns#Mad_Cow_Theatre> <http://example.com/ns#height> "Orlando,_Florida " .

<http://example.com/ns#Kumamoto_Prefectural> <http://example.com/ns#located> "Kumamoto_Prefecture " .

Shapes file: C:\Users\asus\Desktop\nnnnn\msha.ttl

Loaded Data Graph:

<http://example.com/ns#Kumamoto_Prefectural>

<http://example.com/ns#located>

"Kumamoto_Prefecture " .

<http://example.com/ns#Mad_Cow_Theatre>

<http://example.com/ns#height> "Orlando,_Florida " .

<http://example.com/ns#Torres_de_Hercules>

<http://example.com/ns#length> "Los_Barrios " .

<http://example.com/ns#Eccleston_Paddocks>

<http://example.com/ns#latitude>

"Eccleston,_Cheshire " .

<http://example.com/ns#Kumamoto_Prefectural_Theater>

<http://example.com/ns#located>

"Kumamoto_Prefecture " .

Validation report first page

```
Validation report generated.
Validation Report:
PREFIX ex:  <http://example.com/ns#>
PREFIX owl:  <http://www.w3.org/2002/07/owl#>
PREFIX rdf:  <http://www.w3.org/1999/02/22-rdf-syntax-ns#>
PREFIX rdfs: <http://www.w3.org/2000/01/rdf-schema#>
PREFIX sh:  <http://www.w3.org/ns/shacl#>
PREFIX xsd:  <http://www.w3.org/2001/XMLSchema#>

[ rdf:type    sh:ValidationReport;
sh:conforms  false;
sh:result    [ rdf:type                  sh:ValidationResult;
sh:focusNode              ex:Kumamoto_Prefectural;
sh:resultMessage           "Duplicate with the same properties must be marked as owl:sameAs.";
sh:resultSeverity          sh:Violation;
sh:sourceConstraintComponent  sh:SPARQLConstraintComponent;
sh:sourceShape             ex:Shape2;
sh:value                 ex:Kumamoto_Prefectural
];
sh:result    [ rdf:type                  sh:ValidationResult;
sh:focusNode              ex:Kumamoto_Prefectural;
sh:resultMessage           "Duplicate detected for properties: located";
sh:resultSeverity          sh:Violation;
sh:sourceConstraintComponent  sh:SPARQLConstraintComponent;
sh:sourceShape             ex:Shape2;
sh:value                 ex:Kumamoto_Prefectural
];
sh:result    [ rdf:type                  sh:ValidationResult;
sh:focusNode              ex:Kumamoto_Prefectural_Theater;
sh:resultMessage           "Duplicate with the same properties must be marked as owl:sameAs.";
```

Validation report second page

```
sh:resultSeverity          sh:Violation;
sh:sourceConstraintComponent  sh:SPARQLConstraintComponent;
sh:sourceShape              ex:Shape2;
sh:value                  ex:Kumamoto_Prefectural_Theater
];
sh:result    [ rdf:type                  sh:ValidationResult;
sh:focusNode              ex:Kumamoto_Prefectural_Theater;
sh:resultMessage           "Duplicate detected for properties: located";
sh:resultSeverity          sh:Violation;
sh:sourceConstraintComponent  sh:SPARQLConstraintComponent;
sh:sourceShape              ex:Shape2;
sh:value                  ex:Kumamoto_Prefectural_Theater
]
] .
```

Violation Counts by Individual Constraint:

Constraint: http://www.w3.org/ns/shacl#SPARQLConstraintComponent - Duplicate with the same properties must be marked as owl:sameAs.

Violations: 2

Constraint: http://www.w3.org/ns/shacl#SPARQLConstraintComponent - Duplicate detected for properties: located

Violations: 2

Total Number of Violations: 4

Validation report third page

In this example we have two nodes have same values for key islocatedininv, two violations for each node, one for the unique constraint and one for the sameAs rule